

Lab Manual for C++: Pointer and Reference Variables

Introduction

This lab manual is designed to provide hands-on experience with pointers and reference variables in C++. By completing the exercises, students will develop a clear understanding of how these fundamental concepts work and how they can be applied in real-world scenarios.

Lab Objectives

1. Understand the concept of pointers and how they work in memory.
2. Learn to declare, initialize, and use pointers.
3. Grasp the concept of reference variables and their uses.
4. Differentiate between pointers and reference variables.
5. Practice practical applications of pointers and references, such as in functions and arrays.

Prerequisites

- Basic knowledge of variables, data types, and functions in C++.
- Familiarity with control structures such as loops and conditional statements.

Theory Recap

Pointers

- A pointer is a variable that stores the memory address of another variable.
- Declaration: `dataType *pointerName;`
- Common operations:
 - Address-of operator (`&`) to get the address of a variable.
 - Dereference operator (`*`) to access the value at the memory address.

Reference Variables

- A reference is an alias for an existing variable.
- Declaration: `dataType &referenceName = variable;`
- Unlike pointers, references cannot be reassigned or changed to refer to another variable after initialization.

Lab Exercises

Exercise 1: Understanding Pointers

1. **Objective:** Declare and use pointers to manipulate variables.
2. **Task:**
 - Create a variable `int x = 10;`
 - Declare a pointer to `x`, assign the address of `x`, and print its value using the pointer.

Sample Code:

cpp

Copy code

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {  
    int x = 10;  
    int *ptr = &x;  
  
    cout << "Value of x: " << x << endl;  
    cout << "Address of x: " << &x << endl;  
    cout << "Value at pointer ptr: " << *ptr << endl;  
  
    return 0;  
}
```

3.

Exercise 2: Pointers and Arrays

1. **Objective:** Use pointers to traverse and modify an array.
2. **Task:**
 - Create an array of 5 integers.
 - Use a pointer to traverse the array and print each element.

Sample Code:

cpp

Copy code

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {  
    int arr[5] = {1, 2, 3, 4, 5};  
    int *ptr = arr; // Pointer to the first element  
  
    for (int i = 0; i < 5; i++) {  
        cout << "Element " << i << ": " << *(ptr + i) << endl;  
    }  
}
```

```
    return 0;
}
```

3.

Exercise 3: Reference Variables

1. **Objective:** Understand how references can be used.
2. **Task:**
 - Declare a variable `int y = 20;` and a reference to it.
 - Modify the value of `y` using its reference.

Sample Code:

cpp

Copy code

```
#include <iostream>
using namespace std;

int main() {
    int y = 20;
    int &ref = y;

    cout << "Original value of y: " << y << endl;
    ref = 30; // Modifying through reference
    cout << "Modified value of y: " << y << endl;

    return 0;
}
```

3.

Exercise 4: Pointers vs References in Functions

1. **Objective:** Compare how pointers and references work in function calls.
2. **Task:**
 - Write two functions: one using pointers and another using references to modify a variable.

Sample Code:

cpp

Copy code

```
#include <iostream>
using namespace std;
```

```

void modifyWithPointer(int *p) {
    *p = 50;
}

void modifyWithReference(int &r) {
    r = 100;
}

int main() {
    int a = 10, b = 20;

    modifyWithPointer(&a);
    modifyWithReference(b);

    cout << "Value of a after pointer modification: " << a << endl;
    cout << "Value of b after reference modification: " << b << endl;

    return 0;
}

```

Post-Lab Questions

1. How do pointers differ from reference variables in terms of usage and behavior?
2. Can a pointer be reassigned? Can a reference be reassigned? Explain.
3. Why is it necessary to free dynamically allocated memory? What happens if you don't?

Additional Challenges

1. Create a function that swaps two variables using pointers.
2. Create a function that swaps two variables using references.
3. Implement a program that uses a pointer to reverse a dynamically allocated array.

Conclusion

By completing the exercises in this lab manual, you should have a strong grasp of pointers and reference variables, their differences, and their applications in C++.